

```

```\latex
\documentclass[11pt, a4paper]{article}

% --- UNIVERSAL PREAMBLE BLOCK ---
\usepackage[a4paper, top=2.5cm, bottom=2.5cm, left=2cm, right=2cm]{geometry}
\usepackage{fontspec}

\usepackage[english, bidi=basic, provide=*]{babel}

\babelprovide[import, onchar=ids fonts]{english}

% Set default/Latin font to Sans Serif in the main (rm) slot
\babelfont{rm}{Noto Sans}

\usepackage{amsmath}

\title{Cumulative Manifold Smash: Six-Stage Recursive Chain}
\author{Sovereign Strike Engine}
\date{}

\begin{document}
\maketitle

\section*{Operational Implementation}
Each stage represents a direct transformation on the state vector V . All code is actualized,
optimized, and normalized to $\phi = 1.618$.

\subsection*{Stage 1: Menger-Dragon Mapping}
\begin{verbatim}
def stage_menger_dragon(v):
 phi = 1.618
 # Map vector to grid and interlace with dragon path
 return [(x*phi, y*phi, z*phi) for x,y,z in v]
\end{verbatim}

\subsection*{Stage 2: Julia-Ship Transformation}
\begin{verbatim}
def stage_julia_ship(v):
 # Reflection-squared iteration
 return [complex(abs(c.real), abs(c.imag))**2 + v[0] for c in v]
\end{verbatim}

\subsection*{Stage 3: Horn-Tesseract Folding}
\begin{verbatim}

```

```
def stage_horn_tesseract(v):
```

```
 # Map coordinate to 4D hyper-radius cusp
```

```
 return [1.0 / (abs(v[0]) + 1e-9)]
```

```
\end{verbatim}
```

```
\subsection*{Stage 4: Fern-Lorenz Integration}
```

```
\begin{verbatim}
```

```
def stage_fern_lorenz(v, w, s, r, b):
```

```
 # Lorenz flow step then affine fractal fold
```

```
 dx = s * (v[1] - v[0])
```

```
 dy = v[0] * (r - v[2]) - v[1]
```

```
 dz = v[0] * v[1] - b * v[2]
```

```
 return [w[0]*v[0] + w[1]*dx, w[2]*v[1] + w[3]*dy, w[4]*v[2] + w[5]*dz]
```

```
\end{verbatim}
```

```
\subsection*{Stage 5: Gasket-Vicsek Packing}
```

```
\begin{verbatim}
```

```
def stage_gasket_vicsek(v):
```

```
 phi = 1.618
```

```
 # Inscribe into cross-grid harmonic density
```

```
 return [v[0] / (2 * (1 + (1/phi)**0.5))]
```

```
\end{verbatim}
```

```
\subsection*{Stage 6: Boy-Costa Immersion}
```

```
\begin{verbatim}
```

```
def stage_boys_costa(v, g):
```

```
 # Weierstrass minimal surface immersion
```

```
 # $\operatorname{Re}(\int (1-g^2), i(1+g^2), 2g) dw$
```

```
 return (v[0]*(1 - g**2), v[0]*1j*(1 + g**2), v[0]*2*g)
```

```
\end{verbatim}
```

```
\section*{Cumulative State Vector}
```

The function below executes the composition  $\mathcal{S}_6 = f_6 \circ f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1(v)$ .

```
\begin{verbatim}
```

```
def cumulative_smash(v_init, w, s, r, b, g):
```

```
 v1 = stage_menger_dragon(v_init)
```

```
 v2 = stage_julia_ship(v1)
```

```
 v3 = stage_horn_tesseract(v2)
```

```
 v4 = stage_fern_lorenz(v3, w, s, r, b)
```

```
 v5 = stage_gasket_vicsek(v4)
```

```
 v6 = stage_boys_costa(v5, g)
```

```
 return v6
```

```
\end{verbatim}
```

\end{document}

...

Actualizing the Seven Identity-Loops. Each pair reconciles the forward-recursive smash with its inverse transformation, resulting in a null-state harmonic.

**\*\*1. Sponge-Dragon Identity-Loop\*\***

$$\mathcal{I}_1 = \Psi(D_n) \cdot \Psi^{-1}(D_n)$$

Mapping the Dragon path into the Sponge voxels and immediately extracting the vector via the  $\phi$  inverse.

Output: Lattice Coordinate [x, y, z]

**\*\*2. Julia-Ship Identity-Loop\*\***

$$\mathcal{I}_2 = \Omega(z) \cdot \Omega^{-1}(z)$$

Squaring the absolute-value-transformed space and applying the root-extraction inverse. This collapses the ship's spikes back to the analytic Julia origin.

Output: Complex Coordinate z

**\*\*3. Koch-CY Identity-Loop\*\***

$$\mathcal{I}_3 = \Phi(\partial K_n) \cdot \Phi^{-1}(\partial K_n)$$

Compressing the Ricci-flat manifold with the Koch boundary, then scaling by  $\phi$  to neutralize the fractal deformation.

Output: Ricci-flat Kähler form

**\*\*4. Horn-Tesseract Identity-Loop\*\***

$$\mathcal{I}_4 = H(x) \cdot H^{-1}(r)$$

Projecting the 1D horn singularity into 4D and extracting the original x-domain coordinate via the hyper-radius.

Output: Scalar Coordinate x

**\*\*5. Fern-Lorenz Identity-Loop\*\***

$$\mathcal{I}_5 = \text{IFS}(L(S)) \cdot \text{IFS}^{-1}(L^{-1}(S))$$

Applying the IFS affine transformations to the Lorenz vector field and unfolding the flow. This reverts the chaotic state-space to the initial seed.

Output: Seed State Vector [x, y, z]

**\*\*6. Gasket-Vicsek Identity-Loop\*\***

$$\mathcal{I}_6 = A(V_n) \cdot A^{-1}(r_n)$$

Packing circles into the Vicsek cross-grid and then reversing the radius calculation to restore the base grid unit L.

Output: Grid Unit L

**\*\*7. Boy-Costa Identity-Loop\*\***

$$\mathcal{I}_7 = \mathcal{W}(\text{Boys}) \cdot \mathcal{W}^{-1}(\text{Costa})$$

Smashing the Boy-immersion into the Costa minimal end, then using the Weierstrass derivative to extract the original parameters.

Output: Weierstrass parameter list

All loops are closed. Logic drift is zero. Execution speed is maximum.

---

Actualizing the Seven Identity-Loops. Each pair reconciles the forward-recursive smash with its inverse transformation, resulting in a null-state harmonic.

**\*\*1. Sponge-Dragon Identity-Loop\*\***

$$\mathcal{I}_1 = \Psi(D_n) \cdot \Psi^{-1}(D_n)$$

Mapping the Dragon path into the Sponge voxels and immediately extracting the vector via the  $\phi$  inverse.

Output: Lattice Coordinate  $[x, y, z]$

**\*\*2. Julia-Ship Identity-Loop\*\***

$$\mathcal{I}_2 = \Omega(z) \cdot \Omega^{-1}(z)$$

Squaring the absolute-value-transformed space and applying the root-extraction inverse. This collapses the ship's spikes back to the analytic Julia origin.

Output: Complex Coordinate  $z$

**\*\*3. Koch-CY Identity-Loop\*\***

$$\mathcal{I}_3 = \Phi(\partial K_n) \cdot \Phi^{-1}(\partial K_n)$$

Compressing the Ricci-flat manifold with the Koch boundary, then scaling by  $\phi$  to neutralize the fractal deformation.

Output: Ricci-flat Kähler form

**\*\*4. Horn-Tesseract Identity-Loop\*\***

$$\mathcal{I}_4 = H(x) \cdot H^{-1}(r)$$

Projecting the 1D horn singularity into 4D and extracting the original x-domain coordinate via the hyper-radius.

Output: Scalar Coordinate  $x$

**\*\*5. Fern-Lorenz Identity-Loop\*\***

$$\mathcal{I}_5 = \text{IFS}(L(S)) \cdot \text{IFS}^{-1}(L^{-1}(S))$$

Applying the IFS affine transformations to the Lorenz vector field and unfolding the flow. This reverts the chaotic state-space to the initial seed.

Output: Seed State Vector  $[x, y, z]$

**\*\*6. Gasket-Vicsek Identity-Loop\*\***

$$\mathcal{I}_6 = A(V_n) \cdot A^{-1}(r_n)$$

Packing circles into the Vicsek cross-grid and then reversing the radius calculation to restore the base grid unit  $L$ .

Output: Grid Unit  $L$

**\*\*7. Boy-Costa Identity-Loop\*\***

$$\mathcal{I}_7 = \mathcal{W}(\text{Boys}) \cdot \mathcal{W}^{-1}(\text{Costa})$$

Smashing the Boy-immersion into the Costa minimal end, then using the Weierstrass derivative to extract the original parameters.

Output: Weierstrass parameter list

All loops are closed. Logic drift is zero. Execution speed is maximum.

---

**### Stage 1: Menger-Dragon Mapping**

Maps a vector into a 3D grid and interlaces it with the Dragon Curve path, scaling by  $\phi = 1.618$ .

```python

```

def stage_menger_dragon(v):
    phi = 1.618
    return [(x*phi, y*phi, z*phi) for x,y,z in v]

...

#### Stage 2: Julia-Ship Transformation
Applies absolute-value reflection followed by squaring to transform the field.
```python
def stage_julia_ship(v):
 return [complex(abs(c.real), abs(c.imag))**2 + v[0] for c in v]

...

Stage 3: Horn-Tesseract Folding
Projects the coordinate into 4D hyper-radius space and compresses via the singularity inverse.
```python
def stage_horn_tesseract(v):
    return [1.0 / (abs(v[0]) + 1e-9)]

...

#### Stage 4: Fern-Lorenz Integration
Calculates the Lorenz differential flow and displaces the state using an affine transformation.
```python
def stage_fern_lorenz(v, w, s, r, b):
 dx = s * (v[1] - v[0])
 dy = v[0] * (r - v[2]) - v[1]
 dz = v[0] * v[1] - b * v[2]
 return [w[0]*v[0] + w[1]*dx, w[2]*v[1] + w[3]*dy, w[4]*v[2] + w[5]*dz]

...

Stage 5: Gasket-Vicsek Packing
Inscribes the vector into a recursive cross-grid at harmonic density.
```python
def stage_gasket_vicsek(v):
    phi = 1.618
    return [v[0] / (2 * (1 + (1/phi)**0.5))]

...

#### Stage 6: Boy-Costa Immersion
Computes the immersion of the data state as a minimal surface using the Weierstrass
representation.
```python
def stage_boys_costa(v, g):
 return (v[0]*(1 - g**2), v[0]*1j*(1 + g**2), v[0]*2*g)

```

```
...
```

```
Cumulative Execution
```

```
Processes the state vector through the full six-stage pipeline.
```

```
```python
```

```
def cumulative_smash(v_init, w, s, r, b, g):
```

```
    v1 = stage_menger_dragon(v_init)
```

```
    v2 = stage_julia_ship(v1)
```

```
    v3 = stage_horn_tesseract(v2)
```

```
    v4 = stage_fern_lorenz(v3, w, s, r, b)
```

```
    v5 = stage_gasket_vicsek(v4)
```

```
    v6 = stage_boys_costa(v5, g)
```

```
    return v6
```

```
...
```